FTC ROBOTICS TRAINING SERIES

FTC TeleOp

STUDENT WORKBOOK

Seven video-guided lessons for understanding, testing, and improving a two-motor Java TeleOp

1 PROGRAM FLOW	**2** MOTOR MAPPING	**3** INIT / PLAY / STOP	**4** ACTIVE LOOP
5 JOYSTICK VALUES	**6** POWER + TELEMETRY	**7** SPLIT ARCADE	**!** POWERING ON

STUDENT Name: _____ Team: _____

COURSE PROMISE Predict before testing. Record evidence before changing anything. STOP and confirm DISABLED before approaching the robot.

THE ROBOTIC PIT CREW

Build smart. Speak up. Stay safe.

How to use this workbook

STUDENT CYCLE PREDICT -> WATCH AND DISCUSS -> DO THE ACTIVITY -> PASS THE SAFETY GATE -> TEST -> RECORD EVIDENCE -> EXIT TICKET

LESSON	TOPIC	STUDENT PRODUCT	DONE
1	How an FTC TeleOp Program Works	Ordered Human TeleOp flow	[ ]
2	Meet the Motors	Motor mapping and direction record	[ ]
3	INIT, Telemetry, and waitForStart()	INIT-PLAY-STOP observation	[ ]
4	The Active Loop	Traced control-loop cycles	[ ]
5	Reading the Joysticks	Joystick sign-conversion sheet	[ ]
6	Motor Power, Movement, and Telemetry	Power/telemetry evidence table	[ ]
7	Split Arcade Drive	Normalized split-arcade calculations	[ ]

Evidence rules

RULE	WHAT IT LOOKS LIKE
Predict first	Write or say what the code should do before pressing PLAY.
Observe precisely	Record commanded values, telemetry, direction, sound, and stability.
Stop on mismatch	Press STOP immediately when behavior differs from the prediction.
Confirm DISABLED	Look at the Driver Station before anyone crosses the boundary.
Change one thing	Use evidence to choose one check or correction before retesting.

My starting point

WHAT I ALREADY KNOW Write one thing you know about Java or robot driving and one question you want this series to answer.

_____

_____

_____

The POWERING ON safety contract

NON-NEGOTIABLE The sequence is completed before every live-motor test. If a step is skipped, remain disabled and restart at Step 1.

STEP	ACTION	TEAM CHECK
1	SECURE THE ROBOT Use stable blocks. Keep driven wheels and moving mechanisms clear of the table and floor.	[ ]
2	CONFIRM DISABLED Select the correct robot and OpMode, but do not press PLAY.	[ ]
3	LOOK AROUND Remove hands, tools, wires, scraps, and loose objects from moving areas.	[ ]
4	MAKE SURE ALL ARE CLEAR Everyone steps outside the boundary; secure hair, clothing, lanyards, strings, and jewelry.	[ ]
5	CALL AND WAIT Call POWERING ON! loudly. Pause long enough for everyone to stop, look up, and step away.	[ ]
6	VISUALLY CONFIRM, THEN ENABLE Scan the full area again. Press PLAY only when everyone and everything is clear.	[ ]

Callout script

BEFORE PLAY Robot is secured and DISABLED. Look around. Everyone clear? POWERING ON! ... Wait ... Area is visibly clear. Enabling now.

BEFORE CONTACT STOP pressed. Driver Station shows DISABLED. Hands may approach only after the operator confirms disabled.

Student agreement

I understand that anyone may call STOP and the operator will disable immediately. I will not approach or touch the robot until DISABLED is visually confirmed.

Student signature: _____ **Date:** _____

Reference program: BasicTwoMotorTeleOp.java

Use this numbered reference throughout Lessons 1-6. The comments divide the program into startup and repeating control sections.

```
01  package org.firstinspires.ftc.teamcode;
02
03  import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;
04  import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
05  import com.qualcomm.robotcore.hardware.DcMotor;
06
07  @TeleOp(name="Basic Two Motor TeleOp", group="Iterative Opmode")
08  public class BasicTwoMotorTeleOp extends LinearOpMode {
09      private DcMotor leftMotor;
10      private DcMotor rightMotor;
11
12      @Override
13      public void runOpMode() {
14          leftMotor = hardwareMap.get(DcMotor.class, "left_motor");
15          rightMotor = hardwareMap.get(DcMotor.class, "right_motor");
16
17          // Set one motor REVERSE if the physical drivetrain requires it.
18          // rightMotor.setDirection(DcMotor.Direction.REVERSE);
19
20          telemetry.addData("Status", "Initialized");
21          telemetry.update();
22          waitForStart();
23
24          if (opModeIsActive()) {
25              while (opModeIsActive()) {
26                  double leftPower = -gamepad1.left_stick_y;
27                  double rightPower = -gamepad1.right_stick_y;
28                  leftMotor.setPower(leftPower);
29                  rightMotor.setPower(rightPower);
30                  telemetry.addData("Left Power", leftPower);
31                  telemetry.addData("Right Power", rightPower);
32                  telemetry.update();
33              }
34          }
35      }
36  }
```

CODE DETECTIVE Circle waitForStart(). Box the while condition. Underline both setPower() calls. Draw a star beside each telemetry line.

LESSON 1 How an FTC TeleOp Program Works

Become the program and trace the complete path

MISSION Can you move the program token from INIT to STOP without skipping a step?

Learning targets

CHECK	BY THE END OF THIS LESSON, I CAN...
[ ]	Name the program's four main phases.
[ ]	Trace the path from hardware mapping to the active control loop.
[ ]	Explain why the loop repeats until STOP.
[ ]	State the six-step POWERING ON sequence.

Words to know

VOCABULARY OpMode | LinearOpMode | hardwareMap | INIT | PLAY | control loop | telemetry | STOP

Code focus

```
leftMotor = hardwareMap.get(DcMotor.class, "left_motor");
waitForStart();
while (opModeIsActive()) {
   // READ -> COMMAND -> REPORT -> REPEAT
}
```

Before the video: commit to a prediction

MY PREDICTION Circle the phase where you think the program spends most of its driving time: INIT / WAIT / ACTIVE LOOP / STOP

_____

_____

Confidence before the lesson: 1 2 3 4 5 (circle one)

LESSON 1 Guided Video Notes

How an FTC TeleOp Program Works

DIRECTIONS At each timestamp, pause, think silently, discuss, and then record evidence from the code or whiteboard visual.

PAUSE	QUESTION	MY CODE / VISUAL EVIDENCE
0:18	What are the program's four main stations?	
0:47	Do declared motor variables already control physical motors?	
0:58	Why must configuration names match quoted strings exactly?	
1:06	What does waitForStart() do?	
1:26	What must happen before enabling?	
1:37	What four actions repeat inside the loop?	
1:48	Why does the joystick code contain a minus sign?	

MOST IMPORTANT IDEA The idea I would teach to a teammate is:

_____

_____

LESSON 1 Hands-On Team Activity

How an FTC TeleOp Program Works

Materials and setup

READY	ITEM
[ ]	Phase cards: IDENTIFY, CONNECT, WAIT, READ, SET POWER, TELEMETRY, REPEAT, STOP
[ ]	Program token and INIT / PLAY / STOP cards
[ ]	Joystick-value cards and a printed copy of BasicTwoMotorTeleOp.java
[ ]	POWERING ON safety poster (live robot is optional for this activity)

Team roles

ROLE	JOB
Program token	Shows the instruction currently being executed.
Driver Station	Announces INIT, PLAY, and STOP.
Hardware map	Connects Java names to configured devices.
Wait gate	Holds the token until PLAY.
Joystick reader	Supplies a fresh value every cycle.
Motor command	Receives and sends the requested powers.
Telemetry	Reports current commanded values.
Active check	Chooses REPEAT when true and STOP when false.

Activity sequence

STEP	ACTION
1	Arrange IDENTIFY -> CONNECT -> WAIT. Keep the token at WAIT until the Driver Station says PLAY.
2	After PLAY, pass the token through READ -> SET POWER -> TELEMETRY -> REPEAT.
3	Return to the active check. Show TRUE and begin another cycle with a new joystick card.
4	Complete three cycles. Explain why the newest input is used each time.
5	Show STOP/FALSE. The token must exit the loop and move to DISABLED.
6	Point to the safety poster and rehearse the six steps before any future live test.

LESSON 1 — Lab Record and Exit Ticket

How an FTC TeleOp Program Works

Student/team: _____ **Date:** _____ **Robot:** _____

Human TeleOp flow record

TOKEN LOCATION	WHAT HAPPENS	ONCE OR REPEATED?	MY EVIDENCE
IDENTIFY	Find the selected OpMode	Once	
CONNECT	Map Java names to hardware	Once	
WAIT	Pause for PLAY	Once	
READ	Get current joystick values	Repeated	
SET POWER	Send commands to motors	Repeated	
TELEMETRY	Report values	Repeated	
STOP	Exit loop and disable	At stop	

Debrief

#	QUESTION	MY RESPONSE
1	Which actions happen only once before PLAY?	
2	Which actions repeat during TeleOp?	
3	Why does the program return to READ instead of reusing an old value?	
4	What condition sends the token to STOP?	

Exit ticket

ANSWER BEFORE LEAVING Write the complete TeleOp path in one sentence, beginning with INIT and ending with STOP.

_____

_____

LESSON 2 — Meet the Motors

From Java names to real FTC hardware

MISSION Can you prove that each Java motor name reaches the intended physical wheel?

Learning targets

CHECK	BY THE END OF THIS LESSON, I CAN...
[ ]	Separate declaration from hardware mapping.
[ ]	Identify both hardwareMap.get() arguments.
[ ]	Audit exact configuration names.
[ ]	Perform a safe, low-power direction test.

Words to know

VOCABULARY declaration | field | DcMotor | hardwareMap | configuration name | case-sensitive | direction

Code focus

```
private DcMotor leftMotor;
private DcMotor rightMotor;

leftMotor = hardwareMap.get(DcMotor.class, "left_motor");
rightMotor = hardwareMap.get(DcMotor.class, "right_motor");
```

Before the video: commit to a prediction

MY PREDICTION If the configuration says left_motor but the code says leftMotor, predict what will happen during INIT.

_____

_____

Confidence before the lesson: 1 2 3 4 5 (circle one)

LESSON 2 Guided Video Notes

Meet the Motors

DIRECTIONS At each timestamp, pause, think silently, discuss, and then record evidence from the code or whiteboard visual.

PAUSE	QUESTION	MY CODE / VISUAL EVIDENCE
0:25	What does declaring a variable accomplish?	
0:34	Does declaring leftMotor connect it to a physical motor?	
0:45	What job does hardwareMap perform?	
0:56	What are the two hardwareMap.get() arguments?	
1:06	Why must left_motor include the underscore?	
1:23	Why might one drivetrain motor need reversal?	
1:42	What is the safe first-direction-test sequence?	

MOST IMPORTANT IDEA The idea I would teach to a teammate is:

_____

_____

LESSON 2 Hands-On Team Activity

Meet the Motors

Materials and setup

READY	ITEM
[ ]	Robot secured on stable blocks with both drive wheels fully clear.
[ ]	Robot Controller configuration and BasicTwoMotorTeleOp.java displayed side by side.
[ ]	Printed mapping and direction-observation sheet from this guide.
[ ]	Gamepad and Driver Station; POWERING ON safety poster visible at the test boundary.
[ ]	Optional printed mismatch cards: left_motor, leftMotor, Left_Motor, and left motor.

Team roles

ROLE	JOB
Safety lead	Reads each POWERING ON step and watches the boundary.
Driver Station	Selects INIT, PLAY, and STOP only when directed.
Code reader	Finds declarations and hardwareMap.get() lines.
Config navigator	Reads the configured device names exactly.
Left observer	Records left-wheel direction from outside the boundary.
Right observer	Records right-wheel direction from outside the boundary.
Recorder	Completes the mapping and direction tables.
Debugger	Proposes one evidence-based correction after STOP.

Activity sequence

STEP	ACTION
1	Point to private DcMotor leftMotor;. Say: This creates a typed Java name. Has Java connected to the real motor yet?
2	Read each hardwareMap.get() call from the inside out: device type, quoted configuration name, returned motor, assigned variable.
3	Show one incorrect name card. Ask students to predict when the failure occurs and whether PLAY should be pressed.
4	Robot secured. Confirm DISABLED. Look around. Make sure everyone is clear. Call POWERING ON! Wait. Visually confirm clear. Then direct the operator to INIT and PLAY.
5	After STOP and DISABLED, compare commanded side, observed side, configuration name, and direction setting. Change only one cause before retesting.

LESSON 2 — Lab Record and Exit Ticket

Meet the Motors

Student/team: _____ **Date:** _____ **Robot:** _____

BEFORE PLAY	AFTER TEST
[ ] Robot secure [ ] DISABLED [ ] Area clear [ ] POWERING ON called [ ] Wait completed [ ] Visual clear	[ ] STOP pressed [ ] DISABLED visually confirmed Safety lead initials: _____

Motor mapping and direction record

CHECK	LEFT SIDE	RIGHT SIDE	EVIDENCE / NOTES
Java variable	leftMotor	rightMotor	
Device type	DcMotor.class	DcMotor.class	
Quoted code name			Copy exactly
Robot Configuration name			Copy exactly
Exact match?	Yes / No	Yes / No	Circle differences
Gentle forward-stick test	Wheel: ____	Wheel: ____	Expected: ____
Direction correction needed?	Yes / No	Yes / No	Only after DISABLED

Debrief

#	QUESTION	MY RESPONSE
1	What is created by private DcMotor leftMotor;?	
2	What are the two key hardwareMap.get() arguments?	
3	Why do we test one side at a time?	
4	What must happen before anyone approaches after a test?	

Exit ticket

ANSWER BEFORE LEAVING Explain the difference between leftMotor and "left_motor" in one or two sentences.

_____

_____

LESSON 3 — INIT, Telemetry, and waitForStart()

Follow execution through Driver Station states

MISSION Can you identify the exact line where the program waits before PLAY?

Learning targets

CHECK	BY THE END OF THIS LESSON, I CAN…
[ ]	Describe what executes during INIT.
[ ]	Compare telemetry.addData() and update().
[ ]	Explain waitForStart().
[ ]	Describe how PLAY and STOP change execution.

Words to know

VOCABULARY runOpMode | INIT | telemetry packet | update | waitForStart | PLAY | active | STOP

Code focus

```
telemetry.addData("Status", "Initialized");
telemetry.update();

waitForStart();

if (opModeIsActive()) { ... }
```

Before the video: commit to a prediction

MY PREDICTION Before PLAY, will moving the joystick execute the setPower() lines? Explain why or why not.

_____

_____

Confidence before the lesson: 1 2 3 4 5 (circle one)

LESSON 3

Guided Video Notes

INIT, Telemetry, and waitForStart()

DIRECTIONS At each timestamp, pause, think silently, discuss, and then record evidence from the code or whiteboard visual.

PAUSE	QUESTION	MY CODE / VISUAL EVIDENCE
0:17	What causes runOpMode() to begin?	
0:27	What setup happens before waitForStart()?	
0:46	How do addData() and update() differ?	
0:55	Does Initialized mean the driving loop is running?	
1:05	What does waitForStart() wait for?	
1:34	What changes after PLAY?	
1:53	What happens to opModeIsActive() after STOP?	

MOST IMPORTANT IDEA The idea I would teach to a teammate is:

_____

_____

LESSON 3 Hands-On Team Activity

INIT, Telemetry, and waitForStart()

Materials and setup

READY	ITEM
[ ]	Robot secured on blocks; gamepad and Driver Station connected.
[ ]	BasicTwoMotorTeleOp.java displayed with telemetry and waitForStart() visible.
[ ]	Printed state-observation sheet from this guide.
[ ]	Cards labeled BEFORE INIT, INITIALIZED, WAITING, ACTIVE, and STOPPED.
[ ]	POWERING ON safety poster displayed beside the robot area.

Team roles

ROLE	JOB
Safety lead	Controls the call-and-clear sequence.
Driver Station	Presses INIT, PLAY, and STOP when authorized.
Code tracker	Points to the instruction where execution is located.
Telemetry reader	Reads each displayed telemetry message aloud.
Wait gate	Blocks progress until PLAY is pressed.
Joystick tester	Moves one stick only when the teacher authorizes it.
Recorder	Completes the state-observation table.
Stop verifier	Confirms the OpMode ended and the robot is DISABLED.

Activity sequence

STEP	ACTION
1	Place the four state cards in order. Ask students to predict whether motor commands can repeat in each state.
2	With the robot disabled and secured, select the OpMode and press INIT. Ask the code tracker to follow runOpMode() until execution pauses.
3	Ask the joystick tester to move a stick before PLAY while everyone watches from outside the boundary.
4	Run the complete POWERING ON sequence. After the pause and visual clear check, authorize PLAY and one gentle stick movement.
5	Direct the Driver Station operator to press STOP. Ask the stop verifier for two pieces of evidence.

LESSON 3 — Lab Record and Exit Ticket

INIT, Telemetry, and waitForStart()

Student/team: _____ **Date:** _____ **Robot:** _____

BEFORE PLAY	AFTER TEST
[ ] Robot secure [ ] DISABLED [ ] Area clear [ ] POWERING ON called [ ] Wait completed [ ] Visual clear	[ ] STOP pressed [ ] DISABLED visually confirmed Safety lead initials: _____

Program-state observation record

STATE	CODE LOCATION	MOTOR LOOP?	OBSERVED EVIDENCE
Before INIT	Not running	No	
After INIT	Setup -> waitForStart()	No	
Stick moved before PLAY	Still waiting	No	
After PLAY	Active check / while loop	Yes	
Stick returned to center	Loop continues	Yes; power may be 0	
After STOP	Loop exited	No	
DISABLED confirmed	OpMode ended	No	Initials: ____

Debrief

#	QUESTION	MY RESPONSE
1	Why can Initialized be displayed before the robot can drive?	
2	What is the difference between centered sticks and STOP?	
3	What does telemetry.update() do?	
4	What proves it is safe to approach?	

Exit ticket

ANSWER BEFORE LEAVING Why can Status: Initialized be true while joystick movement still produces no wheel response?

_____

_____

LESSON 4 — The Active Loop

Check, repeat, and respond

MISSION Can you trace three loop cycles and stop before a fourth one begins?

Learning targets

CHECK	BY THE END OF THIS LESSON, I CAN...
[ ]	Explain opModeIsActive().
[ ]	Compare if and while checks.
[ ]	Identify the loop body.
[ ]	Explain why fresh input keeps TeleOp responsive.

Words to know

VOCABULARY boolean | true | false | if | while | condition | iteration | loop body | braces

Code focus

```
if (opModeIsActive()) {
   while (opModeIsActive()) {
      // READ
      // COMMAND
      // REPORT
   }
}
```

Before the video: commit to a prediction

MY PREDICTION If opModeIsActive() becomes false at the top of a cycle, which loop statements run next?

_____

_____

Confidence before the lesson: 1 2 3 4 5 (circle one)

LESSON 4 Guided Video Notes

The Active Loop

DIRECTIONS At each timestamp, pause, think silently, discuss, and then record evidence from the code or whiteboard visual.

PAUSE	QUESTION	MY CODE / VISUAL EVIDENCE
0:17	What values can opModeIsActive() return?	
0:26	How often does the outer if check?	
0:36	How often does the while condition check?	
0:45	What do the braces identify?	
1:05	What three jobs happen in each cycle?	
1:44	Why read the joysticks again every cycle?	
1:53	What causes the loop to exit?	

MOST IMPORTANT IDEA The idea I would teach to a teammate is:

_____

_____

LESSON 4 Hands-On Team Activity

The Active Loop

Materials and setup

READY	ITEM
[ ]	Printed BasicTwoMotorTeleOp.java with space to mark braces.
[ ]	Role cards: CHECK, READ, COMMAND, REPORT, DRIVER, CODE TRACER, and STOP.
[ ]	Program token; TRUE/FALSE cards; three left/right joystick-value cards.
[ ]	Optional secured robot and telemetry display for the final extension.
[ ]	POWERING ON safety poster if the live extension will be used.

Team roles

ROLE	JOB
CHECK	Reads opModeIsActive() and chooses repeat or exit.
READ	Accepts the newest joystick-value card.
COMMAND	Sends the current values to the motors.
REPORT	Announces the telemetry values.
Driver	Changes the input card between cycles.
Code tracer	Points to each represented code line.
STOP	Receives the token when the condition is false.
Recorder	Documents each completed iteration.

Activity sequence

STEP	ACTION
1	Ask students to box the condition and connect the opening and closing braces of the while statement.
2	Point to the outer if and inner while. Ask how many times each condition is evaluated when execution reaches it.
3	Give the driver a different input pair before each cycle. Pass the token CHECK -> READ -> COMMAND -> REPORT -> CHECK.
4	Replace TRUE with FALSE when the token returns to CHECK. Ask what happens before another READ.
5	If using the robot, secure it and run the complete POWERING ON sequence before PLAY. Keep sticks centered and observe telemetry changing with a gentle input.

LESSON 4 — Lab Record and Exit Ticket

The Active Loop

Student/team: _____ **Date:** _____ **Robot:** _____

BEFORE PLAY	AFTER TEST
[ ] Robot secure [ ] DISABLED [ ] Area clear [ ] POWERING ON called [ ] Wait completed [ ] Visual clear	[ ] STOP pressed [ ] DISABLED visually confirmed Safety lead initials: _____

Control-loop cycle record

CYCLE	ACTIVE?	NEW INPUT L / R	COMMAND / REPORT / NEXT
1	TRUE	+0.4 / +0.4	Motors: ____ Telemetry: ____ Next: ____
2	TRUE	+0.6 / +0.2	Motors: ____ Telemetry: ____ Next: ____
3	TRUE	0.0 / 0.0	Motors: ____ Telemetry: ____ Next: ____
4	FALSE	New card withheld	Does loop body run? ____ Next: ____
Student-created	____	____ / ____	Prediction: _____

Debrief

#	QUESTION	MY RESPONSE
1	Which jobs repeat?	
2	Why is an active check placed before the body?	
3	What would happen if inputs were read only once?	
4	Does motor power 0.0 end the loop?	

Exit ticket

ANSWER BEFORE LEAVING Describe one full loop cycle and explain exactly what prevents the next cycle after STOP.

_____

_____

LESSON 5 — Reading the Joysticks

Values, variables, and the minus sign

MISSION Can you turn a raw FTC Y value into the correct stored power without guessing?

Learning targets

CHECK	BY THE END OF THIS LESSON, I CAN...
[ ]	Read an assignment from right to left.
[ ]	Explain the double data type.
[ ]	Trace both Y-axis inputs.
[ ]	Negate sample values and predict motion.

Words to know

VOCABULARY double | assignment | expression | gamepad1 | Y axis | raw value | unary minus | tank drive

Code focus

```
double leftPower = -gamepad1.left_stick_y;
double rightPower = -gamepad1.right_stick_y;
```

Before the video: commit to a prediction

MY PREDICTION A forward stick usually reports -0.7. What value is stored after the unary minus is applied?

_____

_____

Confidence before the lesson: 1 2 3 4 5 (circle one)

LESSON 5 Guided Video Notes

Reading the Joysticks

DIRECTIONS At each timestamp, pause, think silently, discuss, and then record evidence from the code or whiteboard visual.

PAUSE	QUESTION	MY CODE / VISUAL EVIDENCE
0:27	What does double tell Java?	
0:36	Which side of an assignment is calculated first?	
0:46	What does gamepad1 identify, and what does Y measure?	
0:56	How does tank drive use the two sticks?	
1:06	What raw value normally means fully forward?	
1:15	What does the minus sign do to that value?	

MOST IMPORTANT IDEA The idea I would teach to a teammate is:

_____

_____

LESSON 5 Hands-On Team Activity

Reading the Joysticks

Materials and setup

READY	ITEM
[ ]	Role cards: RAW VALUE, MINUS SIGN, POWER VARIABLE, and MOTION PREDICTOR.
[ ]	Number cards: -1.0, -0.7, -0.2, 0.0, +0.35, +0.8, and +1.0.
[ ]	Printed joystick-conversion sheet from this guide.
[ ]	BasicTwoMotorTeleOp.java displayed at the leftPower/rightPower assignments.
[ ]	Optional secured robot with power telemetry visible.

Team roles

ROLE	JOB
Left raw value	Displays gamepad1.left_stick_y.
Right raw value	Displays gamepad1.right_stick_y.
Left minus sign	Changes the sign of the left raw value.
Right minus sign	Changes the sign of the right raw value.
Left power	Holds the result stored in leftPower.
Right power	Holds the result stored in rightPower.
Motion predictor	Predicts forward, backward, stop, curve, or spin.
Recorder	Writes raw values, stored values, and predictions.

Activity sequence

STEP	ACTION
1	Display double leftPower = -gamepad1.left_stick_y;. Start at the right side and narrate the order of work.
2	Hand RAW VALUE -0.7 to a student. Ask MINUS SIGN to transform it without changing its magnitude.
3	Give the left and right raw students different cards. Both paths operate at the same time conceptually, but calculate each path clearly.
4	Use raw 0.0 on both sticks. Ask whether the OpMode ends.
5	Secure the robot and complete the POWERING ON sequence. Move one stick gently and compare raw direction, stored sign, and telemetry.

LESSON 5 — Lab Record and Exit Ticket

Reading the Joysticks

Student/team: _____ **Date:** _____ **Robot:** _____

BEFORE PLAY	AFTER TEST
[ ] Robot secure [ ] DISABLED [ ] Area clear [ ] POWERING ON called [ ] Wait completed [ ] Visual clear	[ ] STOP pressed [ ] DISABLED visually confirmed Safety lead initials: _____

Joystick sign-conversion practice

RAW Y VALUE	APPLY -Y	STORED POWER	PREDICTED REQUEST
-1.0	-(-1.0)		
-0.7	-(-0.7)		
0.0	-(0.0)		
+0.35	-(+0.35)		
+1.0	-(+1.0)		
Student value: ____			

Debrief

#	QUESTION	MY RESPONSE
1	Why is Y negated?	
2	What does double communicate?	
3	Are leftPower and rightPower permanent fields here?	
4	Does +0.5 telemetry guarantee physical forward motion?	

Exit ticket

ANSWER BEFORE LEAVING If right_stick_y is +0.35, what is stored in rightPower, and what motion does that side request under the lesson convention?

_____

_____

LESSON 6 — Motor Power, Movement, and Telemetry

Predict, command, observe, and verify

MISSION Can you use code and telemetry to diagnose a mismatch one fact at a time?

Learning targets

CHECK	BY THE END OF THIS LESSON, I CAN...
[ ]	Identify object, method, and argument.
[ ]	Interpret sign and magnitude.
[ ]	Predict tank-drive motion.
[ ]	Compare commands, telemetry, and wheel behavior safely.

Words to know

VOCABULARY object | method | argument | setPower | magnitude | sign | command | telemetry | evidence

Code focus

```
leftMotor.setPower(leftPower);
rightMotor.setPower(rightPower);

telemetry.addData("Left Power", leftPower);
telemetry.addData("Right Power", rightPower);
telemetry.update();
```

Before the video: commit to a prediction

MY PREDICTION Predict the robot request for leftPower = +0.3 and rightPower = -0.3.

_____

_____

Confidence before the lesson: 1 2 3 4 5 (circle one)

LESSON 6 Guided Video Notes

Motor Power, Movement, and Telemetry

DIRECTIONS At each timestamp, pause, think silently, discuss, and then record evidence from the code or whiteboard visual.

PAUSE	QUESTION	MY CODE / VISUAL EVIDENCE
0:28	Identify the object, method, and argument in setPower().	
0:38	What requests stop? What do sign and magnitude mean?	
0:48	What should equal positive side powers do?	
0:58	What should opposite signs do?	
1:08	What does telemetry help the team compare?	
1:47	What should happen when movement differs from prediction?	

MOST IMPORTANT IDEA The idea I would teach to a teammate is:

_____

_____

LESSON 6 Hands-On Team Activity

Motor Power, Movement, and Telemetry

Materials and setup

READY	ITEM
[ ]	Robot secured on stable blocks with both wheels clear.
[ ]	Prediction cards: 0.0/0.0, +0.3/+0.3, -0.3/-0.3, +0.3/-0.3, and +0.5/+0.2.
[ ]	Driver Station showing Left Power and Right Power telemetry.
[ ]	Printed predict-observe-verify sheet from this guide.
[ ]	POWERING ON safety poster visible at the test area.

Team roles

ROLE	JOB
Safety lead	Runs the POWERING ON and approach checkpoints.
Driver Station	Controls INIT, PLAY, and STOP.
Driver	Applies one gentle command at a time.
Left observer	Reports left-wheel direction and relative speed.
Right observer	Reports right-wheel direction and relative speed.
Telemetry reader	Reads commanded Left Power and Right Power.
Recorder	Captures prediction, command, observation, and result.
Debugger	Chooses the next single check after a mismatch.

Activity sequence

STEP	ACTION
1	Display leftMotor.setPower(leftPower); and ask students to identify the object, method, and argument.
2	Reveal one left/right pair at a time. Require a written prediction before any live command.
3	Run the full POWERING ON sequence, press PLAY, move the left stick gently, center it, then move the right stick gently.
4	Use gentle straight, backward, spin, and curve inputs. Pause at center between cases.
5	Press STOP and confirm DISABLED before the team approaches. Choose only one evidence-based check.

LESSON 6 — Lab Record and Exit Ticket

Motor Power, Movement, and Telemetry

Student/team: _____ **Date:** _____ **Robot:** _____

BEFORE PLAY	AFTER TEST
[ ] Robot secure [ ] DISABLED [ ] Area clear [ ] POWERING ON called [ ] Wait completed [ ] Visual clear	[ ] STOP pressed [ ] DISABLED visually confirmed Safety lead initials: _____

Predict - command - observe - verify

LEFT / RIGHT	PREDICTION	TELEMETRY + OBSERVATION	MATCH? / NEXT CHECK
0.0 / 0.0			
+0.3 / +0.3			
-0.3 / -0.3			
+0.3 / -0.3			
+0.5 / +0.2			
Student test: ____ / ____			

Debrief

#	QUESTION	MY RESPONSE
1	What does setPower(0.0) request?	
2	What do sign and magnitude represent?	
3	Why can telemetry and wheel motion disagree?	
4	What is the response to any mismatch?	

Exit ticket

ANSWER BEFORE LEAVING Telemetry shows +0.3 for a wheel that turns physically backward. List the first two facts you would verify after STOP and DISABLED.

_____

_____

LESSON 7 — Split Arcade Drive

One stick drives, one stick turns

MISSION Can you mix and normalize drive and turn while preserving steering?

Learning targets

CHECK	BY THE END OF THIS LESSON, I CAN...
[ ]	Compare tank and split arcade drive.
[ ]	Identify drive and turn axes.
[ ]	Compute left/right mixes.
[ ]	Normalize an out-of-range result.

Words to know

VOCABULARY split arcade | drive | turn | mix | Math.abs | Math.max | denominator | normalize | ratio

Code focus

```
double drive = -gamepad1.left_stick_y;
double turn = gamepad1.right_stick_x;
double denominator = Math.max(Math.abs(drive) + Math.abs(turn), 1.0);
double leftPower = (drive + turn) / denominator;
double rightPower = (drive - turn) / denominator;
```

Before the video: commit to a prediction

MY PREDICTION For drive = +0.5 and turn = +0.5, calculate the raw left and right mixes.

_____

_____

Confidence before the lesson: 1 2 3 4 5 (circle one)

LESSON 7 Guided Video Notes

Split Arcade Drive

DIRECTIONS At each timestamp, pause, think silently, discuss, and then record evidence from the code or whiteboard visual.

PAUSE	QUESTION	MY CODE / VISUAL EVIDENCE
0:18	How do tank and split arcade drive differ?	
0:28	Which axis supplies drive, and why is it negated?	
0:38	Which axis supplies turn?	
0:48	Why does one formula add turn while the other subtracts?	
1:08	How can mixed values exceed the motor-power range?	
1:18	What does normalization accomplish?	

MOST IMPORTANT IDEA The idea I would teach to a teammate is:

_____

_____

LESSON 7 Hands-On Team Activity

Split Arcade Drive

Materials and setup

READY	ITEM
[ ]	BasicArcadeDriveTeleOp.java or the five key split-arcade lines displayed.
[ ]	Role cards: DRIVE, TURN, ADD, SUBTRACT, DENOMINATOR, LEFT POWER, RIGHT POWER, and MOTION.
[ ]	Value cards including 0.0, +0.2, +0.5, +0.6, and +1.0.
[ ]	Printed mix-and-normalize worksheet from this guide.
[ ]	Optional secured robot with Drive, Turn, Left Power, and Right Power telemetry.

Team roles

ROLE	JOB
DRIVE	Holds -gamepad1.left_stick_y.
TURN	Holds gamepad1.right_stick_x.
ADD	Calculates drive + turn.
SUBTRACT	Calculates drive - turn.
DENOMINATOR	Calculates max(abs(drive)+abs(turn), 1.0).
LEFT POWER	Divides the left raw mix by the denominator.
RIGHT POWER	Divides the right raw mix by the denominator.
MOTION	Predicts the resulting robot movement.

Activity sequence

STEP	ACTION
1	Ask students to show with their hands how tank drive and split arcade assign joystick jobs.
2	Set drive = +0.5 and turn = 0.0. Walk through raw left, raw right, denominator, and final outputs.
3	Set drive = 0.0 and turn = +0.5. Require both formulas to be evaluated.
4	Set drive = +1.0 and turn = +1.0. Do not skip the raw-mix step.
5	Students solve drive = +0.6, turn = +0.2. After checking +0.8/+0.4 with denominator 1.0, optionally transfer to the secured robot.

LESSON 7 — Lab Record and Exit Ticket

Split Arcade Drive

Student/team: _____ **Date:** _____ **Robot:** _____

BEFORE PLAY	AFTER TEST
[ ] Robot secure [ ] DISABLED [ ] Area clear [ ] POWERING ON called [ ] Wait completed [ ] Visual clear	[ ] STOP pressed [ ] DISABLED visually confirmed Safety lead initials: _____

Split-arcade mixing and normalization

DRIVE / TURN	RAW LEFT / RIGHT	DENOMINATOR	FINAL L / R + MOTION
+0.5 / 0.0			
0.0 / +0.5			
+0.6 / +0.2			
+1.0 / +1.0			
-0.5 / 0.0			
Student case: ____ / ____			

Debrief

#	QUESTION	MY RESPONSE
1	What are the two driver inputs?	
2	Why can raw mixes reach magnitude 2.0?	
3	What does the denominator guarantee?	
4	What relationship must normalization preserve?	

Exit ticket

ANSWER BEFORE LEAVING For drive = +0.6 and turn = +0.2, calculate leftPower and rightPower and state whether normalization is needed.

_____

_____

FTC TeleOp glossary

TERM	STUDENT-FRIENDLY MEANING
argument	A value supplied inside a method's parentheses.
assignment	Stores the right-side result in the left-side variable.
boolean	A true-or-false value.
configuration name	The exact name assigned to a hardware device on the Robot Controller.
control loop	Repeated code that reads inputs, commands outputs, and reports data.
DcMotor	The FTC SDK type used to control a DC motor.
double	A Java type for decimal/floating-point values.
hardwareMap	The FTC directory that connects configured devices to code.
INIT	Driver Station action that begins OpMode initialization.
iteration	One complete pass through a loop body.
magnitude	The size or strength of a value without considering its sign.
method	An action called on an object, such as setPower().
normalization	Scaling related values together so they fit a required range.
object	A Java value that has data and methods, such as leftMotor.
OpMode	A program that the Driver Station can select and run.
PLAY	Driver Station action that starts the selected initialized OpMode.
telemetry	Data the program sends to the Driver Station for viewing.
unary minus	The minus operator that reverses the sign of one value.
waitForStart	A LinearOpMode method that waits for PLAY or a stop request.
while loop	A loop that repeats while its condition remains true.

My added terms

TERM	MEANING / EXAMPLE

Series capstone: explain and improve the TeleOp

MISSION Use evidence from all seven lessons to explain the current program, demonstrate safe testing, and propose one small, observable improvement.

CHECK	CAPSTONE EVIDENCE
[ ]	Trace INIT -> mapping -> waitForStart -> active loop -> STOP.
[ ]	Identify a Java variable and its exact hardware configuration string.
[ ]	Explain one joystick value from raw input through stored power.
[ ]	Predict a left/right command before testing it.
[ ]	Complete the POWERING ON call-and-clear sequence without prompting.
[ ]	Compare telemetry with observed wheel motion.
[ ]	Calculate one split-arcade mix and explain normalization.

Choose one improvement

CHOICE	WHY IT MAY HELP	MY PICK
Add a joystick deadband	Ignore tiny values near center.	[ ]
Shape joystick response	Give the driver finer low-speed control.	[ ]
Remove the redundant outer if	Simplify code without changing behavior.	[ ]
Compare tank and split arcade	Evaluate which control style fits the driver.	[ ]

MY CLAIM I recommend this change because...

_____

_____

_____

MY TEST The one variable we will change, the prediction, and the evidence we will record:

_____

_____

_____

_____

Engineering notes

Use this page for code sketches, wiring notes, observations, questions, or a controlled-test plan.

FINAL REFLECTION What can you now explain or test that you could not explain or test before Lesson 1?